Reinforcement Learning over Discrete-Time Asset Allocation

MAFS 5370 Reinforcement Learning, Professor Chak Wong
Student: Angeline Candice (20634390)

Introduction

The Setup: You've got a portfolio of up to 4 risky assets (plus cash, because you're not *completely* reckless). Each asset's one-period return follows a normal distribution — rewarding the brave, punishing the foolish.

The Constraint: You can only rebalance 10% of your portfolio per period. Think of it as your broker's way of saying "*calm down.*"

The Catch: You're risk-averse. Not "hide cash under the mattress" risk-averse, but mathematically, provably, *absolutely* risk-averse — via a utility function that makes losses hurt more than gains feel good.

The Goal: Use Reinforcement Learning to find the optimal allocation strategy across any time horizon up to 10 periods, for any $n < 5$ assets, and any reasonable choice of returns, variances, and interest rate r.

The Challenge: The state space is a continuous price vector $\mathbf{X(t)} \in \mathbb{R}^{\mathbf{n}}$ with $\mathbf{X(0) = 1}$. No peeking at the future. Just an agent, a discount rate, and a dream.

Model assumptions

- Assumptions:
  - No short selling
  - No deposits or withdrawals
  - Buy/sell of fractional quantities of assets at any point in time are allowed
  - No transaction costs with any of the buy or sell transactions
  - Constant relative risk aversion

- For analytical tractability, we assume that the portfolio owner is not allowed to short sell cash nor investment asset. Second, the growth of the investment portfolio can happen only from growth in the market value of assets in your investment portfolio.

Symbol table

- t = current time step
- W_t = aggregate market value of your investment assets
- n = fixed number n of risky assets
- $\alpha_1, \ldots, \alpha_n, \alpha_{cash}$ = portfolio weights

Formal definition of the MDP

- STATES
  State = $(t, W_t, \alpha(t))$ where $\alpha(t) = (\alpha_1(t), \ldots, \alpha_n(t), \alpha_{cash}(t))$ is the current allocation vector

- ACTIONS
  Action: $\Delta\alpha(t) = (\Delta\alpha_1, \ldots, \Delta\alpha_n, \Delta\alpha_{cash})$
  Subject to:
  - New allocations: $\alpha'(t) = \alpha(t) + \Delta\alpha(t)$
  - $\alpha'_i(t) \geq 0$ (no short selling)
  - $\Sigma\alpha'_i(t) + \alpha'_{cash}(t) = 1$
  - $\Sigma|\Delta\alpha_i(t)| \leq 0.10$ (or perhaps each $|\Delta\alpha_i| \leq 0.10$, or total rebalancing $\leq$ 10%)

Formal definition of the MDP

- POLICY
  $\pi: (t, W_t, \alpha(t)) \rightarrow \Delta\alpha(t)$

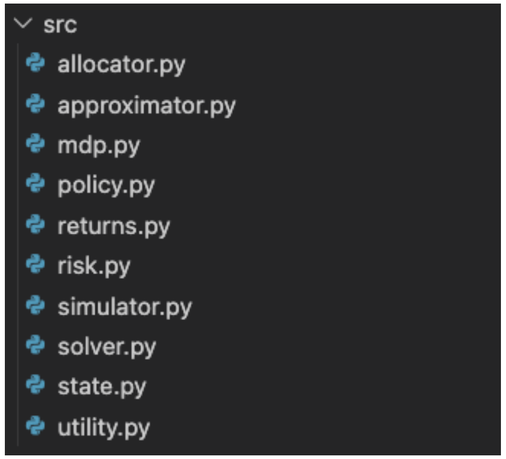
- REWARDS
  Until maturity, there's no intermediate reward since we're not consuming or withdrawing anything — just accumulating wealth. At the final period T, we evaluate the terminal wealth WT using a risk-averse utility function like CRRA

Classes

1. **Utility Function** - A base class with CRRA, log utility, exponential utility.
2. **Return models** - For riskless and risky asset
3. **Risk Aversion** - A base class for risk aversion parameter
4. **State** - Portfolio state (wealth)
5. **Action** - Asset allocation decision
6. **MDP** - The core MDP for asset allocation
7. **Policy** - Optimal policy
8. **Value Function / Q-Value Function approximation**
9. **Backward Induction / RL Solver**
10. **Asset Allocation Engine** - The main orchestrator class

File summary

1. **`utility.py`** — CRRA, Exponential (CARA), and Log utility functions with first-derivative (`marginal`) and risk-aversion property.
2. **`state.py`** — `PortfolioState` (frozen dataclass: wealth, price vector, allocations), `AllocationAction`/`MertonAction` (target allocation dataclasses), and `ActionSpace` (Cartesian grid with rebalancing-limit feasibility checks).
3. **`returns.py`** — Riskless return models (`ConstantRisklessReturn`, `StepwiseRisklessReturn`) and risky return distributions (`NormalReturnDistribution`, `MultivariateNormalReturnDistribution`).
4. **`risk.py`** — Risk aversion schedules: `ConstantRiskAversion` (fixed γ) and `TimeVaryingRiskAversion` (glide-path schedule with terminal-phase clamping).
5. **`mdp.py`** — Abstract `AssetAllocationMDP` and concrete `SingleAssetMDP` / `MultiAssetMDP`. Handles wealth transitions, price evolution, and terminal CRRA reward. Delegates feasibility to `ActionSpace`.
6. **`policy.py`** — `GreedyQPolicy` (argmax over fixed action list), `RandomPolicy`, and `AnalyticalMertonPolicy`(closed-form $\pi^* = (\mu - r)/(\gamma\sigma^2)$, returns unconstrained `MertonAction`).
7. **`approximator.py`** — `LinearQValueApproximator` (ridge regression on feature functions) and `DNNQValueApproximator` (mini-batch SGD MLP with He/Glorot init). Both follow an immutable-update pattern.
8. **`solver.py`** — `BackwardInductionSolver` sweeps $t = T-1 \to 0$, fitting one QVF per step via Bellman targets. Wraps results in `GreedyPolicy`.
9. **`simulator.py`** — `PortfolioSimulator` for full trajectory simulation, `simulate_many`, and `expected_terminal_utility`. `ForwardStateSampler` generates training states by rolling forward with random actions.
10. **`allocator.py`** — `AssetAllocator` top-level orchestrator: wires all components via dependency injection, exposes `run()` → `evaluate_policy()` → `benchmark_against_merton()` → `get_optimal_allocations()`.



Test plan covering our program

- The case where N=1 and T=1
- The case where N=1 and T=10
- The case where N=5 and T=1
- The case where N=5 and T=10

How to start the source code

- To start the project, please run python app.py in the root of the project folder.

```
(.venv) (base) angelinec@Mac ass1 % python app.py
⏳ Pre-computing dashboard data using src classes …
✅ Dashboard data ready.
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [29/Mar/2026 21:35:17] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Mar/2026 21:35:17] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Mar/2026 21:37:58] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Mar/2026 21:37:59] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Mar/2026 21:38:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Mar/2026 21:38:15] "GET / HTTP/1.1" 200 -
```

- A copy of the dashboard is also saved in ./project-1-saved-dashboard.html

Case 1

RL vs Naive Merton — Cases 1–4

● CASE 1 T=1, N=1 | ● CASE 2 T=1, N=4 | ● CASE 3 T=10, N=1 | ● CASE 4 T=10, N=4 | TEST SUITE Unit & Integration | SUMMARY Backward Induction

CASE 1 — T=1, N=1 — Single Period, Single Asset

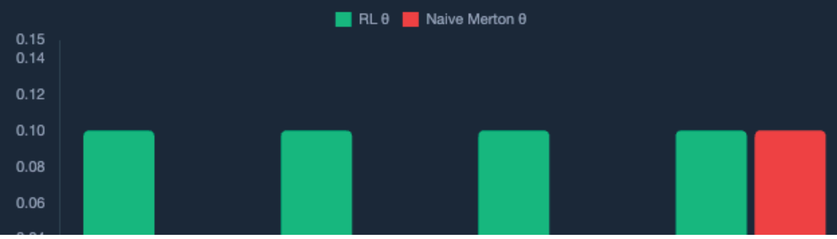
μ=0.06 · r=0.02 · σ=0.20 · W_0=1.0 · REBAL=10 pp · $\gamma \in \{1, 2, 5, 10\}$

RL WINS	NAIVE MERTON FAILURE	TIED CASE	RL OPTIMAL Θ
3/4	All-or-nothing	$\gamma = 10$	0.10
$\gamma \in \{1,2,5\}$	π^*>10 pp, no trade executed	π^*=0.10 exactly = REBAL	Best reachable from θ_0=0

FEASIBILITY ANALYSIS — STARTING ALLOCATION Θ_0 = 0

Γ	MERTON Π* (UNCONSTRAINED)	TRADE REQUIRED |Π*−0|	WITHIN 10% LIMIT?	NAIVE MERTON Θ	RL Θ	WINNER
γ=1	1.00	1.00	✕ No — blocked	0.00	0.10	RL
γ=2	0.50	0.50	✕ No — blocked	0.00	0.10	RL
γ=5	0.20	0.20	✕ No — blocked	0.00	0.10	RL
γ=10	0.10	0.10	✓ Yes	0.10	0.10	Tie

ALLOCATION — RL VS NAIVE MERTON



E[U(W_T)] — RL VS NAIVE MERTON VS CASH

Why are some values negative? CRRA utility is $U(W) = W^{(1-\gamma)}/(1-\gamma)$. For γ>1 this is mathematically always negative — its range is (−∞, 0). A value of −0.5 is better than −1.0. What matters is that RL (green) is always ≥ Naive Merton (red).

Case 2

RL vs Naive Merton — Cases 1–4

CASE 1 T=1, N=1 | CASE 2 T=1, N=4 | CASE 3 T=10, N=1 | CASE 4 T=10, N=4 | TEST SUITE Unit & Integration | SUMMARY Backward Induction

CASE 2 — T=1, N=4 — Single Period, Four Assets

r=0.02 · W_0=1.0 · REBAL=10 pp/asset · Starting allocation θ_0=(0, 0, 0, 0)

RL WINS	RL ALLOCATION	NAIVE MERTON	GATE FAILURE
4/4	(10%×4)	(0%×4)	4× assets
γ=1,2,5 blocked on all 4 assets	Best feasible from θ_0=0	Blocked on all 4 assets simultaneously	Binary gate blocks all four at once

What is the "Merton target"? The Merton formula gives the theoretically ideal allocation for each asset in isolation, assuming no trading constraints: $\pi^*_i = (\mu_i - r)/(\gamma \cdot \sigma_i^2)$. Starting from θ_0=0, reaching any of these targets requires a jump larger than the 10 pp limit. Naive Merton sees this and trades nothing. RL picks the best *reachable* allocation.

ASSET UNIVERSE

ASSET	M	Σ	EXCESS RETURN (M−R)
Asset A	8%	25%	6%
Asset B	6%	20%	4%
Asset C	7%	22%	5%
Asset D	5%	18%	3%

FEASIBILITY ANALYSIS — CAN NAIVE MERTON EXECUTE THE TRADE? (STARTING FROM 0%)

Γ	MERTON TARGET PER ASSET (IDEAL, NO CONSTRAINTS)	SMALLEST JUMP NEEDED BEST CASE |Π*−0|	ALL JUMPS WITHIN 10%? CAN MERTON TRADE?	NAIVE MERTON RESULT WHAT ACTUALLY HAPPENS	RL RESULT BEST REACHABLE	WINNER
γ=1	[96.0%, 100.0%, 103.3%, 92.6%]	92.6%	✕ No — all blocked	(0%, 0%, 0%, 0%)	(10%, 10%, 10%, 10%)	RL
γ=2	[48.0%, 50.0%, 51.7%, 46.3%]	46.3%	✕ No — all blocked	(0%, 0%, 0%, 0%)	(10%, 10%, 10%, 10%)	RL

Case 3

RL vs Naive Merton — Cases 1–4

CASE 1 T=1, N=1 | CASE 2 T=1, N=4 | CASE 3 T=10, N=1 | CASE 4 T=10, N=4 | TEST SUITE Unit & Integration | SUMMARY Backward Induction

CASE 3 — T=10, N=1 — Ten Periods, Single Asset

μ=0.06 · r=0.02 · σ=0.20 · W_0=1.0 · REBAL=10 pp/period · θ_0=0

🏆 **VERDICT:** RL wins for γ=1,2,5. Ties for γ=10 since π*=10% is exactly reachable in one step — both methods land at the same allocation immediately.

RL WINS	NAIVE MERTON	MAX ADVANTAGE	RL STRATEGY
3/4	All-cash	0.143	Ramp-up
γ ∈ {1,2,5}	All 10 periods, γ=1,2,5	γ=1 (log utility)	+10 pp/period toward π*

RL RAMP — PERIODS NEEDED TO REACH Π*

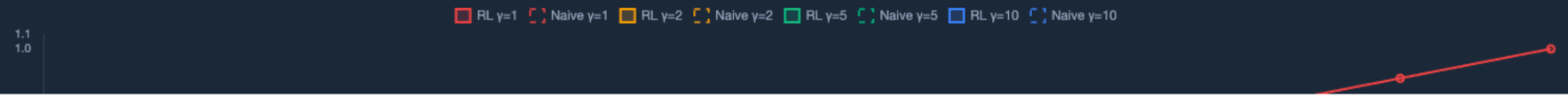
Γ	Π* (MERTON)	STEPS TO REACH Π*	STEPS AT FULL EXPOSURE
γ=1	1.00	10	0
γ=2	0.50	5	5
γ=5	0.20	2	8
γ=10	0.10	1	9

NAIVE MERTON — BLOCKED EVERY PERIOD

Γ	Π*	Θ_OLD EACH PERIOD	JUMP REQUIRED	WITHIN REBAL?	RESULT
γ=1	1.00	0.00	1.00	✗ No	0.00 (blocked)
γ=2	0.50	0.00	0.50	✗ No	0.00 (blocked)
γ=5	0.20	0.00	0.20	✗ No	0.00 (blocked)
γ=10	0.10	0.00	0.10	✓ Yes	0.10

ALLOCATION OVER TIME — RL RAMP VS NAIVE MERTON (ALL Γ)

Solid lines = RL ramp toward π*. Dashed = Naive Merton (stuck at 0 for γ=1,2,5).



Case 4

RL vs Naive Merton — Cases 1–4 | CASE 1 T=1, N=1 | CASE 2 T=1, N=4 | CASE 3 T=10, N=1 | CASE 4 T=10, N=4 | TEST SUITE Unit & Integration | SUMMARY Backward Induction

CASE 4 — T=10, N=4 — Ten Periods, Four Assets

r=0.02 · W_0=1.0 · REBAL=10 pp/period/asset · θ_0=(0, 0, 0, 0)

🏆 **VERDICT:** RL wins for γ=1,2,5. Ties for γ=10. The constraint failure now compounds in two dimensions simultaneously: across 10 time periods and across 4 assets.

RL WINS
4/4
Same γ pattern as Case 3

NAIVE MERTON
Cash
All 10 periods, all 4 assets, γ=1,2,5

MAX ADVANTAGE
0.364
Larger than Case 3 (N=4 amplification)

DOUBLE FAILURE
T × N
10 periods × 4 assets blocked simultaneously

EU SUMMARY — ALL Γ

Γ	RL $E[U(W_T)]$	NAIVE MERTON $E[U(W_T)]$	RL ADVANTAGE	WINNER
γ=1	0.5636	0.2000	+0.3636	RL
γ=2	-0.5986	-0.8187	+0.2201	RL
γ=5	-0.0563	-0.1123	+0.0560	RL
γ=10	-0.0083	-0.0104	+0.0022	RL

ASSET ALLOCATION DETAILS AT END OF HORIZON (T=10)

Merton target π*: the unconstrained ideal (no trading limits assumed) · **Naive Merton:** actual final holding after 10 periods (stays at 0% since all targets exceed 10%) · **RL final:** allocation after ramping 10 pp/period, subject to the no-leverage constraint (sum ≤ 100%)

ASSET	M	Σ	MERTON Π* (Γ=2) NO CONSTRAINTS	NAIVE MERTON (Γ=2) AFTER 10 PERIODS	RL FINAL (Γ=2) AFTER 10 PERIODS	MERTON Π* (Γ=5) NO CONSTRAINTS	NAIVE MERTON (Γ=5) AFTER 10 PERIODS	RL FINAL (Γ=5) AFTER 10 PERIODS